Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Chapter 5: Relational Database Design

Database Systems (CO2013)

Computer Science Program

Assoc. Prof. Dr. Võ Thị Ngọc Châu

(chauvtn@hcmut.edu.vn)

Semester 1 – 2022-2023

Content

- Chapter 1: An Overview of Database Systems
- Chapter 2: The Entity-Relationship Model
- Chapter 3: The Relational Data Model
- Chapter 4: The SQL Language
- **Chapter 5: Relational Database Design**
- Chapter 6: Physical Storage and Data Management
- Chapter 7: Database Security

Chapter 5: Relational Database Design

- 5.1. Design Guidelines for Relation Schemas
- 5.2. Functional Dependencies
- 5.3. Normal Forms Based on Primary Keys
- 5.4. Boyce-Codd Normal Form
- 5.5. Properties of Relational Decompositions
- 5.6. Algorithms for Relational Database Schema Design

Main References

Text:

- [1] R. Elmasri, S. R. Navathe, Fundamentals of Database Systems- 6th Edition, Pearson- Addison Wesley, 2011.
  - ***R. Elmasri, S. R. Navathe, Fundamentals of Database Systems- 7th Edition, Pearson, 2016.***

References:

[1] S. Chittayasothorn, *Relational Database Systems: Language, Conceptual Modeling and Design for Engineers*, Nutcha Printing Co. Ltd, 2017.

[3] A. Silberschatz, H. F. Korth, S. Sudarshan, Database System Concepts – 7th Edition, McGraw-Hill, 2020.

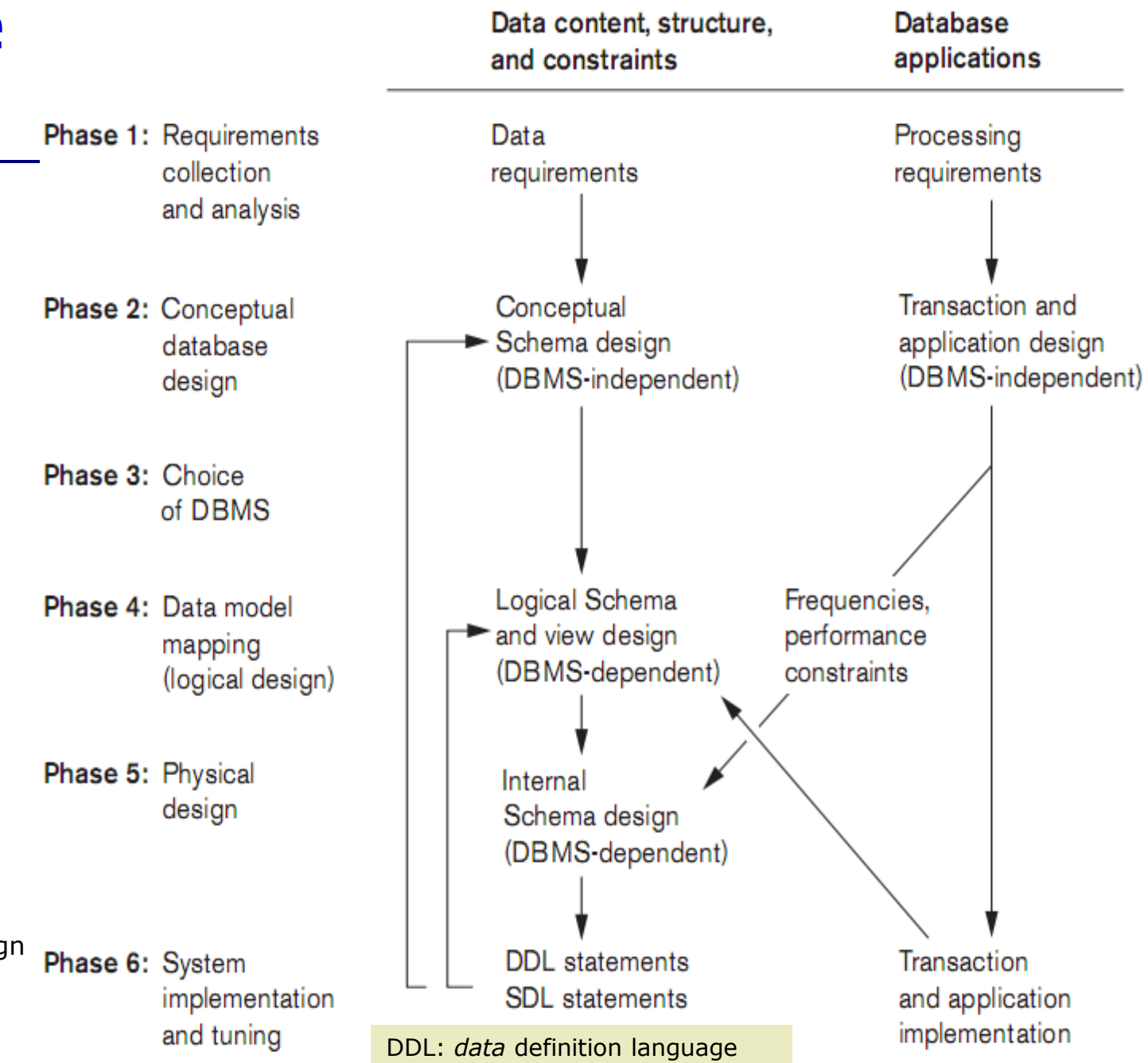
[4] H. G. Molina, J. D. Ullman, J. Widom, *Database Systems: The Complete Book - 2nd Edition,* Prentice-Hall, 2009.

[5] R. Ramakrishnan, J. Gehrke, *Database Management Systems – 4th Edition,* McGraw-Hill, 2018.

[6] M. P. Papazoglou, S. Spaccapietra, Z. Tari, *Advances in Object-Oriented Data Modeling*, MIT Press, 2000.

[7]. G. Simsion, *Data Modeling: Theory and Practice*, Technics Publications, LLC, 2007.

Database design



DDL: *data* definition language
SDL: *storage* definition language

Phases of database design and implementation for large databases

Source: [1]

Relational database design

Two main database design approaches

- A *top-down* design methodology (*design by analysis*)
  - Start with a number of groupings of attributes into relations that exist together naturally (e.g. *invoice*, *report*, and *form*)
  - Analyze the relations individually and collectively, leading to further decomposition until all desirable properties are met
- A *bottom-up* design methodology (*design by synthesis*)
  - Consider the basic relationships among individual attributes as the starting point to construct relation schemas
  - Is not very popular in practice because of the large number of the binary relationships between the attributes

5.1. Design Guidelines for Relation Schemas

- Informal measures of quality for relation schema design
  - *Semantics* of the attributes
    - How to interpret the attribute values stored in a tuple of the relation – how the attribute values in a tuple relate to one another
  - Reducing the *redundant* values in tuples
    - Storage space & update anomalies
  - Reducing the *null* values in tuples
    - Multiple interpretations of *null*s
  - Disallowing the possibility of generating *spurious* tuples
    - Join relations with equality conditions on attributes that are either primary keys or foreign keys in a way that guarantees that no spurious tuples are generated

Semantics of the relation attributes

- Semantics specifies how to *interpret the attribute values stored in a tuple* of the relation, i.e. how the attribute values in a tuple relate to one another.
- It is assumed that attributes belonging to one relation have *certain real-world meaning and a proper interpretation* associated with them.
- If the conceptual design is done carefully, followed by a systematic mapping into relations, *most* of the semantics will have been accounted for and the resulting design should have a clear meaning.
- Design a relation schema so that it is easy to *explain its meaning* and *avoid semantic ambiguities*.
  - Do NOT combine attributes from *multiple* entity types and relationship types into *a single* relation.

Semantics of the relation attributes

EMPLOYEE

Ename	Ssn	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

Better Relations

(*Each tuple captures the details of two real-world entities: employee, department.*)

Worse Relations

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Reducing the *redundant* values in tuples

- Repeated data in one relation
  - More storage space
    - Considered for less execution time of certain queries
  - The anomalies (*Kì dị dữ liệu*)
    - Cause more execution time for insertion, deletion, modification
- Design the base relation schemas so that NO insertion, deletion, or modification anomalies are present in the relations.
  - If any anomalies are present, note them clearly, ensure that the programs that update the database will operate correctly.
- It is advisable to use *anomaly-free base relations* and to specify *views* that include the joins for placing together the attributes frequently referenced in important queries.

Reducing the *redundant* values in tuples

EMPLOYEE

Ename	Ssn	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

Better Relations

Where are redundant values?

Worse Relations

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Reducing the *redundant* values in tuples

Where are redundant values?

Worse Relations

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

John, Carter | 246813579 | 1970-03-01 | 123 Str, Spring, TX | 5

10 | Sales | 987987987

Redundancy

Insertion anomalies:

- Insertion for a *new employee* in an existing department with dnumber=5. Detail of department 5 must be entered again exactly to be consistent with its detail in other existing tuples. Otherwise, *DATA INCONSISTENCY* occurs.

- Insertion for a *new department* with dnumber=10 which has not yet had any employee? IMPOSSIBLE because Ssn is the primary key!!!

Reducing the *redundant* values in tuples

Where are redundant values?

Worse Relations

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy

Deletion anomalies:

- Deletion for the *last single employee* of a department leads to deletion of the details of that department. This makes a *data loss* for departments.
- Delete the tuple with Ssn = 888665555 → delete data of department 1

  → *department 1's data loss*

Reducing the *redundant* values in tuples

Where are redundant values?

Worse Relations

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy

Modification anomalies:

- *Changes on redundant values* lead to update *more than one tuple* to ensure data *consistency*.
- Change Dname from "Research" to "Research & Development" → Update 4 tuples → *data inconsistency* if any of them has not been updated.

Reducing the *null* values in tuples

- Nulls can have multiple interpretations:
  - The attribute does *not apply* to this tuple.
  - The attribute value for this tuple is *unknown*.
  - The value is known but absent as it has *not been recorded* yet.
- The problem of nulls
  - Waste *storage space*
  - Problems with *understanding the meaning of the attributes* and with *specifying JOIN* operations at the logical level.
  - How to account for them when *aggregate operations* such as COUNT or SUM are applied
- Avoid placing attributes in a base relation whose values may frequently be null. If nulls are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

Disallowing the possibility of generating *spurious* tuples

- *Spurious* tuples
  - NOT valid, NOT exist in the mini-world of the database.
  - Generated when the relations from improper design are linked to each other, resulting in undesirable information.
- Design relation schemas so that they can be joined with *equality* conditions on attributes that are either *primary keys or foreign keys* in a way that guarantees that no spurious tuples are generated.
- Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations, because joining on such attributes may produce spurious tuples.

Disallowing the possibility of generating *spurious* tuples

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

The project list

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
453453453	1	20.0
453453453	2	20.0
666884444	3	40.0
888665555	20	NULL
987654321	20	15.0
987654321	30	20.0
987987987	10	35.0
987987987	30	5.0
999887777	10	10.0
999887777	30	30.0

Employees work on projects.

Proper design

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

The project list

EMPLOYEE_PLOCATION

Essn	Plocation
123456789	Bellaire
123456789	Sugarland
333445555	Houston
333445555	Stafford
333445555	Sugarland
453453453	Bellaire
453453453	Sugarland
666884444	Houston
888665555	Houston
987654321	Houston
987654321	Stafford
987987987	Stafford
999887777	Stafford

Employees work on projects at locations of their projects.

IMProper design

Disallowing the possibility of generating *spurious* tuples

PROJECT $\bowtie_{pnumber=pno}$ WORKS_ON

Essn	Pno	Pname	Pnumber	Dnum	Plocation
123456789	1	ProductX	1	5	Bellaire
123456789	2	ProductY	2	5	Sugarland
333445555	2	ProductY	2	5	Sugarland
333445555	3	ProductZ	3	5	Houston
333445555	10	Computerization	10	4	Stafford
333445555	20	Reorganization	20	1	Houston
453453453	1	ProductX	1	5	Bellaire
453453453	2	ProductY	2	5	Sugarland
666884444	3	ProductZ	3	5	Houston
888665555	20	Reorganization	20	1	Houston
987654321	20	Reorganization	20	1	Houston
987654321	30	Newbenefits	30	4	Stafford
987987987	10	Computerization	10	4	Stafford
987987987	30	Newbenefits	30	4	Stafford
999887777	10	Computerization	10	4	Stafford
999887777	30	Newbenefits	30	4	Stafford

Correct results

PROJECT $\bowtie_{plocation=plocation}$ EMPLOYEE_PLOCATION

Essn	Plocation	Pname	Pnumber	Plocation	Dnum
123456789	Bellaire	ProductX	1	Bellaire	5
123456789	Sugarland	ProductY	2	Sugarland	5
333445555	Houston	ProductZ	3	Houston	5
333445555	Houston	Reorganization	20	Houston	1
333445555	Stafford	Computerization	10	Stafford	4
333445555	Stafford	Newbenefits	30	Stafford	4
333445555	Sugarland	ProductY	2	Sugarland	5
453453453	Bellaire	ProductX	1	Bellaire	5
453453453	Sugarland	ProductY	2	Sugarland	5
666884444	Houston	ProductZ	3	Houston	5
666884444	Houston	Reorganization	20	Houston	1
888665555	Houston	ProductZ	3	Houston	5
888665555	Houston	Reorganization	20	Houston	1
987654321	Houston	ProductZ	3	Houston	5
987654321	Houston	Reorganization	20	Houston	1
987654321	Stafford	Computerization	10	Stafford	4
987654321	Stafford	Newbenefits	30	Stafford	4
987987987	Stafford	Computerization	10	Stafford	4
987987987	Stafford	Newbenefits	30	Stafford	4
999887777	Stafford	Computerization	10	Stafford	4
999887777	Stafford	Newbenefits	30	Stafford	4

INCorrect results with SPURIOUS tuples

Summary of design guidelines

- Good database design by minimizing the number of problematic relation schemas
  - *Mixtures* in one single relation
  - *Redundant* data in a relation
  - *Null* values in a relation
  - *Improperly related* relations for joins

Summary of design guidelines

- The problems with problematic relation schemas
  - *Anomalies* that cause *redundant work* to be done during *insertion* into and *modification* of a relation, and that may cause *accidental loss of information during a deletion* from a relation
  - *Waste of storage space* due to *nulls* and the *difficulty* of performing *aggregation* operations (e.g. count, sum, max, min, average) and *joins* due to null values
  - Generation of *invalid and spurious data* during *joins on improperly related base relations*

5.2. Functional Dependencies

- Functional dependency (FD) (*Phụ thuộc hàm*)
  - The single most important concept in relational schema design theory
  - A *constraint* between two sets of attributes from the database
  - A property of the *semantics* of the attributes
    - An FD can*not* be inferred automatically from a given relation extension *r* but *must be defined explicitly* by someone who knows the semantics of the attributes of a relation schema *R*.
    - The *database designers* will use their understanding of the semantics of the attributes of a relation schema *R* - that is, how they relate to one another - to specify the functional dependencies that should *hold on all relation states* (extensions) *r* of *R*.
  - For example: a functional dependency: Pnumber → Plocation
    - The value of a project's number (Pnumber) *uniquely* (functionally) determines the project's location (Plocation).
    - A project's location (Plocation) is functionally dependent on the value of the project's number (Pnumber).
    - There is a functional dependency from Pnumber to Plocation.

5.2. Functional Dependencies

- Functional dependency (FD)
  - A constraint between two sets of attributes from the database
    - A constraint on the possible tuples that can form a relation state *r* of a relation schema $R = \{A_1, A_2, \ldots, A_n\}$
    - Denoted by $X \rightarrow Y$
      - X, Y: two sets of attributes that are subsets of *R*
      - For any two tuples t_1 and t_2 in *r* that have $t_1[X] = t_2[X]$, they must also have $t_1[Y] = t_2[Y]$.

      → The values of Y of a tuple in *r* depend on (are determined by) the values of X.

      → The values of X of a tuple uniquely (functionally) determine the values of Y.

      → If $X \rightarrow Y$ in *R*, this does not say whether or not $Y \rightarrow X$ in *R*.

5.2. Functional Dependencies

TEACH

	STUDENT	COURSE	INSTRUCTOR
	Narayan	Database	Mark
t2	Smith	Database	Navathe
	Smith	Operating Systems	Ammar
	Smith	Theory	Schulman
	Wallace	Database	Mark
	Wallace	Operating Systems	Ahamad
	Wong	Database	Omiecinski
t8	Zelaya	Database	Navathe

A *functional dependency*: Instructor → Course

t2[Instructor] = t8[Instructor] → t2[Course] = t8[Course]

5.2. Functional Dependencies

TEACH

	TEACHER	COURSE	TEXT
t1	Smith	Data Structures	Bartram
t2	Smith	Data Management	Al-Nour
	Hall	Compilers	Hoffman
	Brown	Data Structures	Augenthaler

A *functional dependency*: Text → Course

But Teacher → Course is *NOT* a functional dependency.

t1[Teacher] = t2[Teacher], but t1[Course] ≠ t2[Course]

5.2. Functional Dependencies

- Inference Rules for Functional Dependencies (*Các quy tắc suy diễn cho các phụ thuộc hàm*)
  - An FD $X \rightarrow Y$ is *inferred* from a set of dependencies F specified on R if $X \rightarrow Y$ holds in every legal relation state r of R; that is, whenever r satisfies all the dependencies in F, $X \rightarrow Y$ also holds in r.

  (Phụ thuộc hàm $X \rightarrow Y$ được suy diễn từ tập phụ thuộc F trên lược đồ R nếu $X \rightarrow Y$ đúng với mọi trạng thái quan hệ r của R, nghĩa là: khi r thỏa tất cả các phụ thuộc hàm trong F thì cũng thỏa $X \rightarrow Y$).

  $F \models X \rightarrow Y$ to denote that the functional dependency $X \rightarrow Y$ is *inferred* from the set of functional dependencies F.

  - To determine a systematic way to infer dependencies, a set of *inference rules* can be used to infer new dependencies from a given set of dependencies.

5.2. Functional Dependencies

- Inference Rules for Functional Dependencies
  - IR1 (Reflexive rule, *phản xạ*):

    if $X \supseteq Y$, then $X \rightarrow Y$
  - IR2 (Augmentation rule, *tăng cường*):

    $\{ X \rightarrow Y \} \models XZ \rightarrow YZ$
  - IR3 (Transitive rule, *truyền*):

    $\{ X \rightarrow Y, Y \rightarrow Z \} \models X \rightarrow Z$
  - IR4 (Decomposition, or projective, rule, *chiếu*):

    $\{ X \rightarrow YZ \} \models X \rightarrow Y, X \rightarrow Z$
  - IR5 (Union, or additive, rule, *hợp*):

    $\{ X \rightarrow Y, X \rightarrow Z \} \models X \rightarrow YZ$
  - IR6 (Pseudotransitive rule, *truyền giả*):

    $\{ X \rightarrow Y, WY \rightarrow Z \} \models WX \rightarrow Z$

5.2. Functional Dependencies

- Inference Rules for Functional Dependencies
  - IR1 (Reflexive rule, *phản xạ*):

    if $X \supseteq Y$, then $X \rightarrow Y$
  - IR2 (Augmentation rule, *tăng cường*):

    $\{ X \rightarrow Y \} \models XZ \rightarrow YZ$
  - IR3 (Transitive rule, *truyền*):

    $\{ X \rightarrow Y, Y \rightarrow Z \} \models X \rightarrow Z$

Armstrong's inference rules (axioms, *given facts*):

- *Sound*
- *Complete*

  - IR4 (Decomposition, or projective, rule, *chiếu*):

    $\{ X \rightarrow YZ \} \models X \rightarrow Y, X \rightarrow Z$
  - IR5 (Union, or additive, rule, *hợp*):

    $\{ X \rightarrow Y, X \rightarrow Z \} \models X \rightarrow YZ$
  - IR6 (Pseudotransitive rule, *truyền giả*):

    $\{ X \rightarrow Y, WY \rightarrow Z \} \models WX \rightarrow Z$

Functional Dependencies

- Inference Rules for Functional Dependencies
  - Armstrong's inference rules (axioms, given facts)
    - Sound: Given a set of functional dependencies F specified on a relation schema R, any dependency that can be inferred from F by using IR1 through IR3 holds in every relation state r of R that *satisfies the dependencies* in F.
    - Complete: Using IR1 through IR3 repeatedly to infer dependencies until no more dependencies can be inferred results in the complete set of *all possible dependencies* that can be inferred from F, called the closure of F.

5.2. Functional Dependencies

- The closure of F, the set of functional dependencies specified on relation schema R (*Bao đóng của tập phụ thuộc hàm*)
  - F^+ = set of all dependencies that include *F* and *all dependencies that can be inferred from F*
  - For example: Find the closure of F as follows by using the aforementioned inference rules.

F = { SSN → {ENAME, BDATE, ADDRESS, DNUMBER}, DNUMBER → {DNAME, DMGRSSN} }

F^+ = { SSN → {ENAME, BDATE, ADDRESS, DNUMBER}, DNUMBER → {DNAME, DMGRSSN},
SSN → SSN, ENAME → ENAME, BDATE → BDATE, ADDRESS → ADDRESS,
DNUMBER → DNUMBER, DNAME → DNAME, DMGRSSN → DMGRSSN,
SSN → ENAME, SSN → BDATE, SSN → ADDRESS, SSN → DNUMBER,
DNUMBER → DNAME, DNUMBER → DMGRSSN,
SSN → DNAME, SSN → DMGRSSN, …
}

5.2. Functional Dependencies

- The closure of X under F, where X is the set of attributes of relation schema R

  (*Bao đóng của tập thuộc tính X đối với tập phụ thuộc hàm F*)

  - X = set of *attributes* that appears as a *left-hand side* of some functional dependency in F
  - X^+ = the set of all *attributes* that are dependent on X based on F
  - For example: Find the closure of {SSN} under F as follows by using the aforesaid inference rules.

F = {SSN → {ENAME, BDATE, ADDRESS, DNUMBER}, DNUMBER → {DNAME, DMGRSSN}}

{ SSN } $^+$ = {SSN, ENAME, BDATE, ADDRESS, DNUMBER, DNAME, DMGRSSN}

5.2. Functional Dependencies

An algorithm to determine X^+ under F:

Determining X^+, the Closure of X under F

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

$X^+ := X$;
repeat
    old$X^+ := X^+$;
    for each functional dependency $Y \rightarrow Z$ in F do
        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;
until (X^+ = oldX^+);

F = { SSN → ENAME, PNUMBER → {PNAME, PLOCATION}, {SSN, PNUMBER} → HOURS }

{ SSN } $^+$ = { SSN, ENAME }

{ PNUMBER } $^+$ = { PNUMBER, PNAME, PLOCATION }

{ SSN, PNUMBER } $^+$ = { SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS }

5.2. Functional Dependencies

- Closure X^+ of a set of attributes X under F where X is a set of attributes that appears as a left-hand side of a functional dependency in F
  - $F = \{ AB \rightarrow C, A \rightarrow DE, B \rightarrow M, M \rightarrow GH, D \rightarrow IJ \}$
  - $\{A, B\}^+ = ???$
  - $\{A\}^+ = ???$
  - $\{B\}^+ = ???$
  - $\{M\}^+ = ???$
  - $\{D\}^+ = ???$

5.2. Functional Dependencies

- Closure X^+ of a set of attributes X under F where X is a set of attributes that appears as a left-hand side of a functional dependency in F
  - $F = \{ AB \rightarrow C, A \rightarrow DE, B \rightarrow M, M \rightarrow GH, D \rightarrow IJ \}$
  - $\{A, B\}^+ = \{A, B, C, D, E, M, G, H, I, J\}$
  - $\{A\}^+ = \{A, D, E, I, J\}$
  - $\{B\}^+ = \{B, M, G, H\}$
  - $\{M\}^+ = \{M, G, H\}$
  - $\{D\}^+ = \{D, I, J\}$

5.2. Functional Dependencies

- Superkey (*siêu khóa*) of *a relation schema R*: a *subset* of attributes with the property that no two tuples in any relation state r of R should have the same combination of values for these attributes.
  - One *default superkey* = the *set of all its attributes*
- Key (*khóa*) of a relation schema R (K): a superkey of R with the additional property that removing any attribute A from K leaves a set of attributes K' that is not a superkey of R.
  - Key is called *minimal superkey* (*siêu khóa tối tiểu*).
- Candidate key (*khóa dự tuyển*): any *key* of R.
- Primary key (*khóa chính, khóa sơ cấp*): one of the candidate keys of R is used for implementation.
- Secondary key (*khóa phụ, khóa thứ cấp*): the remaining candidate keys of R with NOT NULL and UNIQUE constraints.
- Prime attribute (*thuộc tính nguyên tố*): a member (attribute) of *some candidate key* of R.

5.2. Functional Dependencies

An algorithm to determine *one* key K of relation schema R:

Finding a Key K for R Given a Set F of Functional Dependencies

Input: A relation R and a set of functional dependencies F on the attributes of R.

1. Set $K := R$.
2. For each attribute A in K
   {compute $(K - A)^+$ with respect to F;
   if $(K - A)^+$ contains all the attributes in R, then set $K := K - \{A\}$ };

F = { SSN → ENAME, PNUMBER → {PNAME, PLOCATION}, {SSN, PNUMBER} → HOURS }

K = { SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS }

$(K - \{SSN\})^+$ = { ENAME, PNUMBER, PNAME, PLOCATION, HOURS }

$(K - \{ENAME\})^+$ = { SSN, PNUMBER, PNAME, PLOCATION, HOURS, ENAME } = R ⇒ K = (K – ENAME)

$(K - \{PNUMBER\})^+$ = { SSN, PNAME, PLOCATION, HOURS, ENAME }

$(K - \{PNAME\})^+$ = { SSN, PNUMBER, PLOCATION, HOURS, ENAME, PNAME } = R ⇒ K = (K – PNAME)

$(K - \{PLOCATION\})^+$ = {SSN, PNUMBER, HOURS, ENAME, PNAME, PLOCATION} = R ⇒ K = (K – PLOCATION)

$(K - \{HOURS\})^+$ = {SSN, PNUMBER, ENAME, PNAME, PLOCATION, HOURS} = R ⇒ K = (K – HOURS)

Key K = {SSN, PNUMBER}

5.2. Functional Dependencies

- Identifying a key K of R based on a set of given functional dependencies F
  - R = (A, B, C, D, E, G)
  - F = { C → E, A → EG, E → AD }
  - K = ???

5.2. Functional Dependencies

Identifying **a key K of R** based on a set of given functional dependencies F

- R = (A, B, C, D, E, G)
- F = {C $\rightarrow$ E, A $\rightarrow$ EG, E $\rightarrow$ AD}
- K = {A, B, C, D, E, G}
- (K-{A})+ = {B, C, D, E, G, A} $\Rightarrow$ K = {B, C, D, E, G}
- (K-{B})+ = {C, D, E, G, A} $\Rightarrow$ K = {B, C, D, E, G}
- (K-{C})+ = {B, D, E, G, A} $\Rightarrow$ K = {B, C, D, E, G}
- (K-{D})+ = {B, C, E, G, A, D} $\Rightarrow$ K = {B, C, E, G}
- (K-{E})+ = {B, C, G, E, A, D} $\Rightarrow$ K = {B, C, G}
- (K-{G})+ = {B, C, E, A, D, G} $\Rightarrow$ **K = {B, C}**

5.2. Functional Dependencies

- Identifying **a key K of R** based on a set of given functional dependencies F
  - R = (A, B, C, E, G)
  - F = {A $\rightarrow$ B, BC $\rightarrow$ E, EG $\rightarrow$ A}
  - K = ???

5.2. Functional Dependencies

- Identifying **a key K of R** based on a set of given functional dependencies F
  - R = (A, B, C, E, G)
  - F = {A $\rightarrow$ B, BC $\rightarrow$ E, EG $\rightarrow$ A}
  - K = ???

K = {A, B, C, E, G}

$(K-\{A\})^+ = \{B, C, E, G, A\} = R \Rightarrow K = \{B, C, E, G\}$

$(K-\{B\})^+ = \{C, E, G, A, B\} = R \Rightarrow K = \{C, E, G\}$

$(K-\{C\})^+ = \{E, G, A, B\} \subset R \Rightarrow K = \{C, E, G\}$

$(K-\{E\})^+ = \{C, G\} \subset R \Rightarrow K = \{C, E, G\}$

$(K-\{G\})^+ = \{C, E\} \subset R \Rightarrow \mathbf{K = \{C, E, G\}}$

5.2. Functional Dependencies

- Identifying all keys K of R based on a set of given functional dependencies F
  - Find $N = U - \cup_{\forall f \in F} \text{right}(f)$

    N is a set of the attributes not in the right side of any functional dependency in F. U is a set of all the attributes of R.
  - Find N^+, the closure of N under F
  - If $N^+ = U$ then there is only one key K = N.
  - Otherwise, find $D = \cup_{\forall f \in F} \text{right}(f) - \cup_{\forall f \in F} \text{left}(f)$

    D is a set of the attributes only in the right side of the functional dependencies in F.
  - Find $L = U - (N^+ \cup D)$

    L is a set of the attributes neither in the right side nor functionally determined by N.
  - Let L_i be any subset of L. Find $(N \cup L_i)^+$ under F.
  - If $(N \cup L_i)^+ = U$ then $K = N \cup L_i$.
  - $\forall K_i, K_j$ such that $K_i \subset K_j$, remove K_j.

5.2. Functional Dependencies

- Identifying **all keys K of R** based on a set of given functional dependencies F
  - R = (A, B, C, E, G)
  - F = {A $\rightarrow$ B, BC $\rightarrow$ E, EG $\rightarrow$ A}
  - All keys K ???

5.2. Functional Dependencies

- $R = (A, B, C, E, G)$
- $F = \{A \rightarrow B, BC \rightarrow E, EG \rightarrow A\}$
- $N = R - \cup_{\forall f \in F} \text{right}(f)$
- $N = \{A, B, C, E, G\} - \{A, B, E\} = \{C, G\}$
- $N^+ = \{C, G\}$
- $D = \cup_{\forall f \in F} \text{right}(f) - \cup_{\forall f \in F} \text{left}(f)$
- $D = \{A, B, E\} - \{A, B, C, E, G\} = \varnothing$
- $L = R - (N^+ \cup D) = \{A, B, E\}$
- Subsets of L: $L_1 = \{A\}$, $L_2 = \{B\}$, $L_3 = \{E\}$, $L_4 = \{A, B\}$, $L_5 = \{A, E\}$, $L_6 = \{B, E\}$, $L_7 = \{A, B, E\}$
- $(N \cup L_1)^+ = \{C, G, A, B, E\} \Rightarrow \mathbf{K_1 = \{C, G, A\}}$
- $(N \cup L_2)^+ = \{C, G, B, E, A\} \Rightarrow \mathbf{K_2 = \{C, G, B\}}$
- $(N \cup L_3)^+ = \{C, G, E, A, B\} \Rightarrow \mathbf{K_3 = \{C, G, E\}}$

5.2. Functional Dependencies

- Identifying all keys K of R based on a set of given functional dependencies F
  - R = (A, B, C, D, E, G)
  - F = { C → E, A → EG, E → AD }
  - K = ???

5.2. Functional Dependencies

Identifying **all keys K of R** based on a set of given functional dependencies F

- R = (A, B, C, D, E, G)
- F = {C $\rightarrow$ E, A $\rightarrow$ EG, E $\rightarrow$ AD}
- N = R - $\cup_{\forall f \in F}$right(f)
- N = {A, B, C, D, E, G} – {A, D, E, G} = {B, C}
- N^+ = {B, C, E, A, D, G} = U
- Only one key K = N = **{B, C}**

5.2. Functional Dependencies

- Identifying **all keys K of R** based on a set of given functional dependencies F
  - R = (A, B, C, E, G)
  - F = {A $\rightarrow$ BC, CG $\rightarrow$ E, B $\rightarrow$ G, E $\rightarrow$ A}
  - All keys K ???

5.2. Functional Dependencies

- Identifying **all keys K of R** based on a set of given functional dependencies F
  - R = (A, B, C, E, G)
  - F = {A → BC, CG → E, B → G, E → A}
  - All keys K ???
- N = R - $\cup_{\forall f \in F}$right(f)
- N = {A, B, C, E, G} – {B, C, E, G, A} = ∅
- D = $\cup_{\forall f \in F}$right(f) - $\cup_{\forall f \in F}$left(f)
- D = {B, C, E, G, A} – {A, C, G, B, E} = ∅
- L = U - ($N^+ \cup D$) = {A, B, C, E, G}
- Subsets of L: L_1 = {A}, L_2 = {B}, L_3 = {C}, L_4 = {E}, L_5 = {G}, L_6 = {A, B}, L_7 = {A, C}, L_8 = {A, E}, L_9 = {A, G}, L_{10} = {B, C}, L_{11} = {B, E}, L_{12} = {B, G}, L_{13} = {C, E}, L_{14} = {C, G}, L_{15} = {E, G}, L16 = {A, B, C}, L_{17} = {A, B, E}, L_{18} = {A, B, G}, L_{19} = {A, C, E}, L_{20} = {A, C, G}, L_{21} = {B, C, E}, L22 = {B, C, G}, L_{23} = {A, E, G}, L_{24} = {B, E, G}, L_{25} = {C, E, G}, L_{26} = {A, B, C, E}, L_{27} = {A, B, C, G}, L_{28} = {A, B, E, G}, L_{29} = {A, C, E, G}, L_{30} = {B, C, E, G}, L_{31} = {A, B, C, E, G}
- $(N \cup L_1)^+$ = {A, B, C, G, E} = R ⇒ **K = {A}**
- $(N \cup L_2)^+$ = {B, G} ⊂ R
- $(N \cup L_3)^+$ = {C} ⊂ R
- $(N \cup L_4)^+$ = {E, A, B, C, G} = R ⇒ **K = {E}**
- $(N \cup L_5)^+$ = {G} ⊂ R
- $(N \cup L_{10})^+$ = {B, C, G, E, A} = R ⇒ **K = {B, C}**
- $(N \cup L_{12})^+$ = {B, G} ⊂ R
- $(N \cup L_{14})^+$ = {C, G, E, A, B} = R ⇒ **K = {C, G}**

5.2. Functional Dependencies

- $F = \{A \rightarrow C,\ AC \rightarrow D,\ E \rightarrow AD,\ E \rightarrow H\}$
- $R = \{A, C, D, E, H\}$
- Identify **a key** of R?

$K = \{A, C, D, E, H\}$

$(K-\{A\})+ = \{C, D, E, H, A\} = R \Rightarrow K = \{C, D, E, H\}$

$(K-\{C\})+ = \{D, E, H, A, C\} = R \Rightarrow K = \{D, E, H\}$

$(K-\{D\})+ = \{E, H, A, D, C\} = R \Rightarrow K = \{E, H\}$

$(K-\{E\})+ = \{H\} \subset R \Rightarrow K = \{E, H\}$

$(K-\{H\})+ = \{E, A, D, H, C\} = R \Rightarrow K = \{E\}$

5.2. Functional Dependencies

- $F = \{A \rightarrow C,\ AC \rightarrow D,\ E \rightarrow AD,\ E \rightarrow H\}$
- $R = \{A, C, D, E, H\}$
- Identify **all the keys** of R?

$N = R - \cup_{\forall f \in F} right(f) = \{A, C, D, E, H\} - \{C, D, A, H\}$

$N = \{E\}$

$N+ = \{E, A, D, H, C\} = R \Rightarrow K = \{E\}$

5.2. Functional Dependencies

- **Membership** of a functional dependency $X \rightarrow Y$ with respect to a set of functional dependencies F, i.e. $X \rightarrow Y$ is inferred from F, denoted by $F \models X \rightarrow Y$

  - Check if $F \models X \rightarrow Y$
    - Find X^+, the closure of X under F
    - If $Y \subseteq X^+$ then $F \models X \rightarrow Y$

5.2. Functional Dependencies

- **Membership** of a functional dependency $X \rightarrow Y$ with respect to a set of functional dependencies F, i.e. $X \rightarrow Y$ is inferred from F, denoted by $F \models X \rightarrow Y$
  - $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$
  - $F \models A \rightarrow CD$
    - A^+ under $F = \{A, C, D\} \supseteq \{C, D\} \Rightarrow F \models A \rightarrow CD$
  - $F \models E \rightarrow AH$???

5.2. Functional Dependencies

- **Membership** of a functional dependency $X \rightarrow Y$ with respect to a set of functional dependencies F, i.e. $X \rightarrow Y$ is inferred from F, denoted by $F \models X \rightarrow Y$
  - $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$
  - $F \models A \rightarrow CD$
    - A^+ under $F = \{A, C, D\} \supseteq \{C, D\} \Rightarrow F \models A \rightarrow CD$
  - $F \models E \rightarrow AH$???
    - E^+ under $F = \{E, A, D, H, C\} \supseteq \{A, H\} \Rightarrow F \models E \rightarrow AH$

5.2. Functional Dependencies

- A set of functional dependencies F is said to **cover** another set of functional dependencies E if every FD in E is also in F^+; that is, if every dependency in E can be inferred from F. Alternatively, E is **covered by** F.
- Two sets of functional dependencies E and F are **equivalent** if $E^+ = F^+$; that is, every dependency in E can be inferred from F and every dependency in F can be inferred from E; that is also, E covers F and F covers E.

5.2. Functional Dependencies

- $F = \{A \rightarrow C,\ AC \rightarrow D,\ E \rightarrow AD,\ E \rightarrow H\}$
- $G = \{A \rightarrow CD,\ E \rightarrow AH\}$

Are F and G **equivalent**?

* Check if F **covers** G (*F phủ G*).

* Check if G **covers** F (*G phủ F*).

5.2. Functional Dependencies

- $F = \{A \rightarrow C,\ AC \rightarrow D,\ E \rightarrow AD,\ E \rightarrow H\}$
- $G = \{A \rightarrow CD,\ E \rightarrow AH\}$

Are F and G **equivalent**?

* Check if F **covers** G (*F phủ G*).

$F \models A \rightarrow CD$

A^+ under $F = \{A, C, D\} \supseteq \{C, D\}$

$F \models E \rightarrow AH$

E^+ under $F = \{E, A, D, H, C\} \supseteq \{A, H\}$

$\Rightarrow$ F covers G.

5.2. Functional Dependencies

- $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$
- $G = \{A \rightarrow CD, E \rightarrow AH\}$

Are F and G **equivalent**?

* Check if G **covers** F (*G phủ F*).

$G \models A \rightarrow C$

A^+ under G = $\{A, C, D\} \supseteq \{C\}$

$G \models AC \rightarrow D$

$\{A, C\}^+$ under G = $\{A, C, D\} \supseteq \{D\}$

$G \models E \rightarrow AD$

E^+ under G = $\{E, A, H, C, D\} \supseteq \{A, D\}$

$G \models E \rightarrow H$

E^+ under G = $\{E, A, H, C, D\} \supseteq \{H\}$

$\Rightarrow$ G covers F. In the previous slide, F covers G.

$\Rightarrow$ **F and G are equivalent.**

5.2. Functional Dependencies

- The **minimal cover** F (*phủ tối tiểu F*) of a set of functional dependencies E is a set of functional dependencies that satisfies the property that every dependency in E is in the closure F^+ of F. In addition, this property is lost if any dependency from the set F is removed; F must have no redundancies in it, and the dependencies in F are in a standard form.

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E

1. Every dependency in F has a single attribute for its right-hand side.
2. We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X, and still have a set of dependencies that is equivalent to F.
3. We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F.

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E

Finding a Minimal Cover F for a Set of Functional Dependencies E

1. Set $F := E$.
2. Replace each functional dependency $X \rightarrow \{A_1, A_2, \ldots, A_n\}$ in F by the n functional dependencies $X \rightarrow A_1, X \rightarrow A_2, \ldots, X \rightarrow A_n$.
3. For each functional dependency $X \rightarrow A$ in F

   for each attribute B that is an element of X

   if $\{ \{ F - \{X \rightarrow A\} \} \cup \{(X - \{B\}) \rightarrow A\} \}$ is equivalent to F,

   then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F.
4. For each remaining functional dependency $X \rightarrow A$ in F

   if $\{ F - \{X \rightarrow A\} \}$ is equivalent to F,

   then remove $X \rightarrow A$ from F.

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E
  - E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}
  - The minimal cover F of E ???

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E
  - E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}
  - **The minimal cover F of E** ???

E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}

F = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

Check for ABCD → I by removing A:

F = {A → B, BCD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

E ⊭ BCD → I: {B, C, D}+ under E = {B, C, D} ⊅ {I}

F ⊨ ABCD → I: {A, B, C, D}+ under F = {A, B, C, D, I} ⊇ {I} ⇒ always true

Check for ABCD → I by removing B:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

E ⊨ ACD → I: {A, C, D}+ under E = {A, C, D, B, I} ⊇ {I}

F ⊨ ABCD → I: always true

Check for ACD → I by removing C:

F = {A → B, AD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

E ⊭ AD → I: {A, D}+ under E = {A, D, B} ⊅ {I}

F ⊨ ACD → I: always true

Check for ACD → I by removing D:

F = {A → B, AC → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

E ⊭ AC → I: {A, C}+ under E = {A, C, B} ⊅ {I}

F ⊨ ACD → I: always true

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E
  - $E = \{A \rightarrow B, ABCD \rightarrow I, IJ \rightarrow G, IJ \rightarrow H, ACDJ \rightarrow GI\}$
  - **The minimal cover F of E** ???

$E = \{A \rightarrow B, ABCD \rightarrow I, IJ \rightarrow G, IJ \rightarrow H, ACDJ \rightarrow GI\}$

$F = \{A \rightarrow B, ACD \rightarrow I, IJ \rightarrow G, IJ \rightarrow H, ACDJ \rightarrow G, ACDJ \rightarrow I\}$

Check for $IJ \rightarrow G$ by removing I:

$F = \{A \rightarrow B, ACD \rightarrow I, J \rightarrow G, IJ \rightarrow H, ACDJ \rightarrow G, ACDJ \rightarrow I\}$

$E \nvDash J \rightarrow G$: $\{J\}+$ under $E = \{J\} \not\supseteq \{G\}$

$F \nvDash IJ \rightarrow G$: $\{I, J\}+$ under $F = \{I, J, G, H\} \supseteq \{G\} \Rightarrow$ always true

Check for $IJ \rightarrow G$ by removing J:

$F = \{A \rightarrow B, ACD \rightarrow I, I \rightarrow G, IJ \rightarrow H, ACDJ \rightarrow G, ACDJ \rightarrow I\}$

$E \nvDash I \rightarrow G$: $\{I\}+$ under $E = \{I\} \not\supseteq \{G\}$

$F \nvDash IJ \rightarrow G$: always true

Check for $IJ \rightarrow H$ by removing I:

$F = \{A \rightarrow B, ACD \rightarrow I, IJ \rightarrow G, J \rightarrow H, ACDJ \rightarrow G, ACDJ \rightarrow I\}$

$E \nvDash J \rightarrow H$: $\{J\}+$ under $E = \{J\} \not\supseteq \{H\}$

$F \nvDash IJ \rightarrow H$: always true

Check for $IJ \rightarrow H$ by removing J:

$F = \{A \rightarrow B, ACD \rightarrow I, IJ \rightarrow G, I \rightarrow H, ACDJ \rightarrow G, ACDJ \rightarrow I\}$

$E \nvDash I \rightarrow H$: $\{I\}+$ under $E = \{I\} \not\supseteq \{H\}$

$F \nvDash IJ \rightarrow H$: always true

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E
  - E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}
  - **The minimal cover F of E** ???

E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

Check for ACDJ → G by removing A:

F = {A → B, ACD → I, IJ → G, IJ → H, CDJ → G, ACDJ → I}

E ⊨ CDJ → G: {C, D, J}+ under E = {C, D, J} ⊉ {G}

F ⊨ ACDJ → G: {A, C, D, J}+ under F = {A, C, D, J, B, I, G, H} ⊇ {G} ⇒ always true

Check for ACDJ → G by removing C:

F = {A → B, ACD → I, IJ → G, IJ → H, ADJ → G, ACDJ → I}

E ⊨ ADJ → G: {A, D, J}+ under E = {A, D, J, B} ⊉ {G}

F ⊨ ACDJ → G: always true

Check for ACDJ → G by removing D:

F = {A → B, ACD → I, IJ → G, IJ → H, ACJ → G, ACDJ → I}

E ⊨ ACJ → G: {A, C, J}+ under E = {A, C, J, B} ⊉ {G}

F ⊨ ACDJ → G: always true

Check for ACDJ → G by removing J:

F = {A → B, ACD → I, IJ → G, IJ → H, ACD → G, ACDJ → I}

E ⊨ ACD → G: {A, C, D}+ under E = {A, C, D, B, I} ⊉ {G}

F ⊨ ACDJ → G: always true

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E
  - E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}
  - **The minimal cover F of E** ???

E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

Check for ACDJ → I by removing A:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, CDJ → I}

E ⊨ CDJ → I: {C, D, J}+ under E = {C, D, J} ⊉ {G}

F ⊨ ACDJ → I: {A, C, D, J}+ under F = {A, C, D, J, B, I, G, H} ⊇ {I} ⇒ always true

Check for ACDJ → I by removing C:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ADJ → I}

E ⊨ ADJ → I: {A, D, J}+ under E = {A, D, J, B} ⊉ {I}

F ⊨ ACDJ → I: always true

Check for ACDJ → I by removing D:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ACJ → I}

E ⊨ ACJ → I: {A, C, J}+ under E = {A, C, J, B} ⊉ {I}

F ⊨ ACDJ → I: always true

Check for ACDJ → I by removing J:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G}

E ⊨ ACD → I: {A, C, D}+ under E = {A, C, D, B, I} ⊇ {I}

F ⊨ ACDJ → I: always true

5.2. Functional Dependencies

- The **minimal cover** F of a set of functional dependencies E
  - E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}
  - **The minimal cover F of E** ???

E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ACDJ → I}

Check for ACDJ → I by removing A:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, CDJ → I}

E ⊨ CDJ → I: {C, D, J}+ under E = {C, D, J} ⊉ {G}

F ⊨ ACDJ → I: {A, C, D, J}+ under F = {A, C, D, J, B, I, G, H} ⊇ {I} ⇒ always true

Check for ACDJ → I by removing C:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ADJ → I}

E ⊨ ADJ → I: {A, D, J}+ under E = {A, D, J, B} ⊉ {I}

F ⊨ ACDJ → I: always true

Check for ACDJ → I by removing D:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G, ACJ → I}

E ⊨ ACJ → I: {A, C, J}+ under E = {A, C, J, B} ⊉ {I}

F ⊨ ACDJ → I: always true

Check for ACDJ → I by removing J:

F = {A → B, ACD → I, IJ → G, IJ → H, ACDJ → G}

E ⊨ ACD → I: {A, C, D}+ under E = {A, C, D, B, I} ⊇ {I}

F ⊨ ACDJ → I: always true

Functional Dependencies

- The minimal cover F of a set of functional dependencies E
  - E={A→B,ABCD→I,IJ→G,IJ→H,ACDJ→GI}
  - The minimal cover F of E ???

Continue till you obtain the final minimal cover F:

$$F = \{A \rightarrow B,\ ACD \rightarrow I,\ IJ \rightarrow G,\ IJ \rightarrow H\}$$

5.2. Functional Dependencies

$F = \{A \rightarrow B,\ ACD \rightarrow I,\ IJ \rightarrow G,\ IJ \rightarrow H,\ ACDJ \rightarrow G\}$

$R = \{A, B, C, D, G, H, I, J\}$

Identify *a* key of R???

$K = \{A, B, C, D, G, H, I, J\}$

$(K-\{A\})+ = \{B, C, D, G, H, I, J\} \supseteq R \Rightarrow K = \{A, B, C, D, G, H, I, J\}$

$(K-\{B\})+ = \{A, C, D, G, H, I, J, B\} = R \Rightarrow K = \{A, C, D, G, H, I, J\}$

$(K-\{C\})+ = \{A, D, G, H, I, J, B\} \supseteq R \Rightarrow K = \{A, C, D, G, H, I, J\}$

$(K-\{D\})+ = \{A, C, G, H, I, J, B\} \supseteq R \Rightarrow K = \{A, C, D, G, H, I, J\}$

$(K-\{G\})+ = \{A, C, D, H, I, J, B, G\} = R \Rightarrow K = \{A, C, D, H, I, J\}$

$(K-\{H\})+ = \{A, C, D, I, J, B, G, H\} = R \Rightarrow K = \{A, C, D, I, J\}$

$(K-\{I\})+ = \{A, C, D, J, B, I, G, H\} = R \Rightarrow K = \{A, C, D, J\}$

$(K-\{J\})+ = \{A, C, D, B, I\} \supseteq R \Rightarrow$ **$K = \{A, C, D, J\}$**

5.2. Functional Dependencies

$F = \{A \rightarrow B,\ ACD \rightarrow I,\ IJ \rightarrow G,\ IJ \rightarrow H,\ ACDJ \rightarrow G\}$

$R = \{A, B, C, D, G, H, I, J\}$

Identify *all* keys of R???

$N = R - \cup_{\forall f \in F} \text{right}(f) = \{A, B, C, D, G, H, I, J\} - \{B, I, G, H\} = \{A, C, D, J\}$

$N^+ = \{A, C, D, J, B, I, G, H\} = R$

R has *only one* key K = N = **{A, C, D, J}**.

5.3. Normal Forms Based on Primary Keys

- Normalization (*Chuẩn hóa*)
  - A process of analyzing the given relation schemas based on their FDs and (*primary*) key*s* to achieve the desirable properties of:

    (1) minimizing redundancy

    (2) minimizing the insertion, deletion, and update anomalies
- Normal form of a relation

  (*Dạng chuẩn của một quan hệ*)
  - The highest normal form condition that the relation meets → the degree to which it has been normalized

5.3. Normal Forms Based on Primary Keys

- The process of normalization through decomposition produces the relational schemas that have:
  - The lossless join (*kết không có mất thông tin*) or non-additive join (*kết không có thêm thông tin*) property

    (*Đặc tính bảo toàn thông tin*)
    - No spurious tuple generation problem occurs.
  - The dependency preservation property

    (*Đặc tính bảo toàn phụ thuộc hàm*)
    - Each FD is represented in some individual relation.

5.3. Normal Forms Based on Primary Keys

Normal forms based on primary keys and corresponding normalization

⇒ *Generalized* to all the ***keys*** for the primary key, ***prime*** attributes for the key attributes, and ***nonprime*** attributes for the nonkey attributes

NORMAL FORM	TEST	REMEDY (NORMALIZATION)
First (1NF)	Relation should have no nonatomic attributes or nested relations.	Form new relations for each nonatomic attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes.) That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

Source: [1]

5.3. Normal Forms Based on Primary Keys

- Pure relations based on the relational data model are all in *1NF*.
  - Atomic attributes*: KHÔNG có nhóm lặp hay quan hệ lồng.*
- Relations in *2NF* have non-key attributes functionally dependent on the *whole key*.
  - Atomic attributes*: KHÔNG có nhóm lặp hay quan hệ lồng.*
  - *Thuộc tính không khóa KHÔNG có phụ thuộc hàm riêng phần vào khóa.*
- Relations in *3NF* have non-key attributes *only* functionally dependent on the whole key.
  - Atomic attributes*: Không có nhóm lặp hay quan hệ lồng.*
  - *Thuộc tính không khóa KHÔNG có phụ thuộc hàm riêng phần vào khóa.*
  - *Thuộc tính không khóa KHÔNG có phụ thuộc hàm bắc cầu vào khóa.*

5.3. Normal Forms Based on Primary Keys

DLOCATIONS: non-atomic attribute (*repeating group*)

DEPARTMENT

DNAME	DNUMBER	DMGRSSN	DLOCATIONS
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

Unnormalized table

1NF relation

DEPARTMENT

DNAME	DNUMBER	DMGRSSN	DLOCATION
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

5.3. Normal Forms Based on Primary Keys

EMP_PROJ

SSN	ENAME	PROJS	
		PNUMBER	HOURS
123456789	Smith,John B.	1	32.5
		2	7.5
666884444	Narayan,Ramesh K.	3	40.0
453453453	English,Joyce A.	1	20.0
		2	20.0
333445555	Wong,Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0

PROJS: non-atomic attribute (*composite*, *repeating group*)

Unnormalized table

EMP_PROJ1

SSN	ENAME
123456789	Smith,John B.
666884444	Narayan,Ramesh K.
453453453	English,Joyce A.
333445555	Wong,Franklin T.

EMP_PROJ2

SSN	PNUMBER	HOURS
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0

1NF relations

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 1NF relations?

1NF relation

DEPARTMENT

DNAME	DMGRSSN	DNUMBER	DLOCATION
Research	333445555	5	Bellaire
Research	333445555	5	Sugarland
Research	333445555	5	Houston
Administration	987654321	4	Stafford
Headquarters	888665555	1	Houston

Primary key: {DNUMBER, DLOCATION}

Functional dependencies:

{DNUMBER, DLOCATION} → {DNAME, DMGRSSN}

DNUMBER → {DNAME, DMGRSSN}

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 1NF relations?

1NF relation

DEPARTMENT

DNAME	DMGRSSN	DNUMBER	DLOCATION
Research	333445555	5	Bellaire
Research	333445555	5	Sugarland
Research	333445555	5	Houston
Administration	987654321	4	Stafford
Headquarters	888665555	1	Houston

Redundancy

Primary key: {DNUMBER, DLOCATION}

Functional dependencies:

{DNUMBER, DLOCATION} → {DNAME, DMGRSSN}

DNUMBER → {DNAME, DMGRSSN} ⇒ *partially* functionally dependent on the key

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 1NF relations?

1NF relation

DEPARTMENT

DNAME	DMGRSSN	DNUMBER	DLOCATION
Research	333445555	5	Bellaire
Research	333445555	5	Sugarland
Research	333445555	5	Houston
Administration	987654321	4	Stafford
Headquarters	888665555	1	Houston

Redundancy

Insertion anomalies: (4, Houston)? (Sales, 123456789, 10)?

Deletion anomalies: (1, Houston)? (Research, 333445555, 5)?

Modification anomalies: DMGRSSN of Department 5 to 123456789?

DNAME of Department 5 to "Research & Development"?

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 1NF relations?

NORMALIZATION to 2NF relations

1NF relation

DEPARTMENT

DNAME	DMGRSSN	DNUMBER	DLOCATION
Research	333445555	5	Bellaire
Research	333445555	5	Sugarland
Research	333445555	5	Houston
Administration	987654321	4	Stafford
Headquarters	888665555	1	Houston

Primary key: {DNUMBER, DLOCATION}

Functional dependencies:

{DNUMBER, DLOCATION} → {DNAME, DMGRSSN}

DNUMBER → {DNAME, DMGRSSN} ⇒ *partially* functionally dependent on the key ⇒ new relation

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 1NF relations?

NORMALIZATION to 2NF relations

1NF relation

DEPARTMENT

DNAME	DMGRSSN	DNUMBER	DLOCATION
Research	333445555	5	Bellaire
Research	333445555	5	Sugarland
Research	333445555	5	Houston
Administration	987654321	4	Stafford
Headquarters	888665555	1	Houston

2NF relations

DEPARTMENT

DNAME	DMGRSSN	DNUMBER
Research	333445555	5
Administration	987654321	4
Headquarters	888665555	1

DEPARTMENT_LOCATION

DNUMBER	DLOCATION
5	Bellaire
5	Sugarland
5	Houston
4	Stafford
1	Houston

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 2NF relations?

EMP_DEPT

ENAME	SSN	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5	Research	333445555
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4	Administration	987654321
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4	Administration	987654321
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5	Research	333445555
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5	Research	333445555
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4	Administration	987654321
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1	Headquarters	888665555

Primary key: SSN

Functional dependencies:

$$SSN \rightarrow \{ENAME, BDATE, ADDRESS, DNUMBER\}$$

$$DNUMBER \rightarrow \{DNAME, DMGRSSN\}$$

Atomic attributes, no partial dependency on the key $\Rightarrow$ 2NF relation

5.3. Normal Forms Based on Primary Keys

Redundancy

EMP_DEPT

ENAME	SSN	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5	Research	333445555
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4	Administration	987654321
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4	Administration	987654321
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5	Research	333445555
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5	Research	333445555
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4	Administration	987654321
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1	Headquarters	888665555

Primary key: SSN

ANY PROBLEMS?

Functional dependencies:

$SSN \rightarrow \{ENAME, BDATE, ADDRESS, DNUMBER\}$

$DNUMBER \rightarrow \{DNAME, DMGRSSN\}$

$\Rightarrow$ *transitively* functionally dependent on the key

Atomic attributes, no partial dependency on the key $\Rightarrow$ 2NF relation

5.3. Normal Forms Based on Primary Keys

Redundancy

EMP_DEPT

ENAME	SSN	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5	Research	333445555
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4	Administration	987654321
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4	Administration	987654321
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5	Research	333445555
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5	Research	333445555
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4	Administration	987654321
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1	Headquarters	888665555

Insertion anomalies: Employee 999777555 in Department 4?

Department 10 with name Sales?

Deletion anomalies: Employee 888665555? Department 5?

Modification anomalies: DMGRSSN of Department 4 to 123456789?

DNAME of Department 5 to "Research & Development"?

Atomic attributes, no partial dependency on the key ⇒ **2NF relation**

5.3. Normal Forms Based on Primary Keys

Redundancy

EMP_DEPT

ENAME	SSN	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5	Research	333445555
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4	Administration	987654321
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4	Administration	987654321
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5	Research	333445555
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5	Research	333445555
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4	Administration	987654321
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1	Headquarters	888665555

ANY PROBLEMS?

NORMALIZATION to 3NF

Primary key: SSN

Functional dependencies:

SSN $\rightarrow$ {ENAME, BDATE, ADDRESS, DNUMBER}

DNUMBER $\rightarrow$ {DNAME, DMGRSSN}

$\Rightarrow$ *transitively* functionally dependent on the key

$\Rightarrow$ new relation

Atomic attributes, no partial dependency on the key $\Rightarrow$ 2NF relation

5.3. Normal Forms Based on Primary Keys

EMP_DEPT

ENAME	SSN	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5	Research	333445555
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4	Administration	987654321
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4	Administration	987654321
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5	Research	333445555
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5	Research	333445555
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4	Administration	987654321
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1	Headquarters	888665555

2NF relation

3NF relations

EMP_DEPT

ENAME	SSN	BDATE	ADDRESS	DNUMBER
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1

DEPARTMENT

DNUMBER	DNAME	DMGRSSN
5	Research	333445555
4	Administration	987654321
1	Headquarters	888665555

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 3NF relations?

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

Candidate key = {STUDENT, COURSE}

FD1: {STUDENT, COURSE} → INSTRUCTOR

FD2: INSTRUCTOR → COURSE

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 3NF relations?

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

Redundancy

Candidate key = {STUDENT, COURSE}

FD1: {STUDENT, COURSE} → INSTRUCTOR

FD2: INSTRUCTOR → COURSE

⇒ *a prime attribute is functionally dependent on a NONprime attribute.*

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 3NF relations?

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

Redundancy

Insertion anomalies: Elmasri teaches Database?

Deletion anomalies: Wong studies Database?

Modification anomalies: Mark teaches Algorithms instead of Database?

5.3. Normal Forms Based on Primary Keys

What *problems* exist with 3NF relations?

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

Redundancy

NORMALIZATION

BCNF relations

Candidate key = {STUDENT, COURSE}

FD1: {STUDENT, COURSE} → INSTRUCTOR

FD2: INSTRUCTOR → COURSE

⇒ *a prime attribute is functionally dependent on a NONprime attribute.*

5.4. Boyce-Codd Normal Form

Boyce-Codd normal form (BCNF)

- A simpler form of 3NF, but stricter than 3NF
- Every relation in BCNF is also in 3NF; however, a relation in 3NF is not necessarily in BCNF.
  - For a relation in 3NF, each non-key attribute must:
    - Fully functionally dependent on every key
    - NOT transitively dependent on every key
      - Not functionally dependent on other nonkey attributes
  - For a relation in BCNF, all key/non-key attributes must be *functionally dependent on superkeys*.
    - Every $X \rightarrow A$ holds in R and X is a superkey. $\Rightarrow$ R in BCNF.
    - Any $X \rightarrow A$ holds in R and X is not a superkey.

    $\Rightarrow$ R is not in BCNF.

5.4. Boyce-Codd Normal Form

3NF relation

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

Candidate key = {STUDENT, COURSE}

FD1: {STUDENT, COURSE} → INSTRUCTOR

FD2: INSTRUCTOR → COURSE

Normalization to BCNF ???

DECOMPOSITION 1

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

BCNF relations

STUDENT_COURSE

STUDENT	COURSE
Narayan	Database
Smith	Database
Smith	Operating Systems
Smith	Theory
Wallace	Database
Wallace	Operating Systems
Wong	Database
Zelaya	Database

INSTRUCTOR_COURSE

COURSE	INSTRUCTOR
Database	Mark
Database	Navathe
Operating Systems	Ammar
Theory	Schulman
Operating Systems	Ahamad
Database	Omiecinski

DECOMPOSITION 1

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

BCNF relations

STUDENT_COURSE

STUDENT	COURSE
Narayan	Database
Smith	Database
Smith	Operating Systems
Smith	Theory
Wallace	Database
Wallace	Operating Systems
Wong	Database
Zelaya	Database

INSTRUCTOR_COURSE

COURSE	INSTRUCTOR
Database	Mark
Database	Navathe
Operating Systems	Ammar
Theory	Schulman
Operating Systems	Ahamad
Database	Omiecinski

How about FD1?

FD1: {STUDENT, COURSE} → INSTRUCTOR

DECOMPOSITION 2

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

BCNF relations

STUDENT_INSTRUCTOR

STUDENT	INSTRUCTOR
Narayan	Mark
Smith	Navathe
Smith	Ammar
Smith	Schulman
Wallace	Mark
Wallace	Ahamad
Wong	Omiecinski
Zelaya	Navathe

INSTRUCTOR_COURSE

COURSE	INSTRUCTOR
Database	Mark
Database	Navathe
Operating Systems	Ammar
Theory	Schulman
Operating Systems	Ahamad
Database	Omiecinski

DECOMPOSITION 2

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

3NF relation

BCNF relations

STUDENT_INSTRUCTOR

STUDENT	INSTRUCTOR
Narayan	Mark
Smith	Navathe
Smith	Ammar
Smith	Schulman
Wallace	Mark
Wallace	Ahamad
Wong	Omiecinski
Zelaya	Navathe

INSTRUCTOR_COURSE

COURSE	INSTRUCTOR
Database	Mark
Database	Navathe
Operating Systems	Ammar
Theory	Schulman
Operating Systems	Ahamad
Database	Omiecinski

How about FD1?

FD1: {STUDENT, COURSE} → INSTRUCTOR

5.5. Properties of Relational Decompositions

- Given a relation schema R and a set F of functional dependencies that should hold on the attributes of R specified by the database designers, a ***decomposition*** of R is a set of relation schemas $D = \{R_1, R_2, \ldots, R_m\}$ where *no attributes are lost* (the *attribute preservation condition*).

$$\bigcup_{i=1}^{m} R_i = R$$

5.5. Properties of Relational Decompositions

- Properties of relational decompositions
  - Dependency preservation property of a decomposition

    (*Phân rã bảo toàn phụ thuộc hàm*)
  - Lossless (nonadditive) join property of a decomposition

    (*Phân rã bảo toàn thông tin*)

5.5. Properties of Relational Decompositions

- Dependency preservation property of a decomposition
  - The dependency preservation condition: each functional dependency $X \rightarrow Y$ specified in F either **appeared directly** in one of the relation schemas R_i in the decomposition D or could be **inferred** from the dependencies that appear in some R_i.
  - If a decomposition is not dependency-preserving, some dependency is ***lost*** in the decomposition.
    - To check that a ***lost*** dependency holds, the JOIN of two or more relations in the decomposition must be performed to get a relation that includes all left- and right-hand-side attributes of the lost dependency, and then check that the dependency holds on the result of the JOIN.

5.5. Properties of Relational Decompositions

- Dependency preservation property of a decomposition
  - Given a set of dependencies F on R, the projection of F on R , denoted by $\pi_{R_i}(F)$ where R_i is a subset of R, is the set of dependencies X $\rightarrow$ Y in F^+ such that the attributes in X U Y are all contained in R_i. Hence, the projection of F on each relation schema R_i in the decomposition D is the set of functional dependencies in F^+, the closure of F, such that all their left- and right-hand-side attributes are in R_i. A decomposition D = {R_1, R_2, ..., R_m} of R is ***dependency-preserving*** *with respect to F* if the union of the projections of F on each R_i in D is equivalent to F; that is,

$$\left((\pi_{R_1}(F)) \cup \cdots \cup (\pi_{R_m}(F))\right)^+ = F^+$$

5.5. Properties of Relational Decompositions

- Dependency preservation property of a decomposition
  - A decomposition D = {R_1, R_2, ..., R_m} of R is ***dependency-preserving*** *with respect to F* if the union of the projections of F on each R_i in D is equivalent to F; that is,

$$\left((\pi_{R_1}(F)) \cup \cdots \cup (\pi_{R_m}(F))\right)^+ = F^+$$

In the previous examples, do their decompositions possess the *dependency preservation property*?

5.5. Properties of Relational Decompositions

- Dependency preservation property of a decomposition $\left((\pi_{R_1}(F)) \cup \cdots \cup (\pi_{R_m}(F))\right)^+ = F^+$

In the previous examples, do their decompositions possess the *dependency preservation property*?

BEFORE

- Relation: TEACH (STUDENT, COURSE, INSTRUCTOR)
  - Primary key: STUDENT, COURSE
  - Functional dependencies
    - FD1: {STUDENT, COURSE} → INSTRUCTOR
    - FD2: INSTRUCTOR → COURSE

DECOMPOSITION 1

- Relation: STUDENT_COURSE (STUDENT, COURSE)
  - Primary key: STUDENT, COURSE
- Relation: COURSE_INSTRUCTOR (COURSE, INSTRUCTOR)
  - Primary key: INSTRUCTOR
  - Functional dependency: INSTRUCTOR → COURSE

DECOMPOSITION 2

- Relation: STUDENT_INSTRUCTOR (STUDENT, INSTRUCTOR)
  - Primary key: STUDENT, INSTRUCTOR
- Relation: COURSE_INSTRUCTOR (COURSE, INSTRUCTOR)
  - Primary key: INSTRUCTOR
  - Functional dependency: INSTRUCTOR → COURSE

5.5. Properties of Relational Decompositions

- Lossless (nonadditive) join property of a decomposition
  - No ***spurious*** tuples are generated when a NATURAL JOIN operation is applied to the relations in the decomposition.
  - A decomposition D = {R_1, R_2, ... , R_m} of R has the ***lossless (nonadditive) join*** property *with respect to the set of dependencies F on R* if, for every relation state *r* of R that satisfies F, the following holds, where * is the NATURAL JOIN of all the relations in D:

$$*\left(\pi_{R_1}(r), \ldots, \pi_{R_m}(r)\right) = r$$

5.5. Properties of Relational Decompositions

- Lossless (nonadditive) join property of a decomposition
  - **Test** the *lossless* (*nonadditive*) *join* property of a *binary decomposition*

A decomposition D = $\{R_1, R_2\}$ of R has the *lossless* (*nonadditive*) *join* property *with respect to a set of functional dependencies F* on R if and only if *either*:

- The FD $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+, *or*
- The FD $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+.

5.5. Properties of Relational Decompositions

Test the *lossless* (*nonadditive*) *join* property of a *binary decomposition*

A decomposition $D = \{R_1, R_2\}$ of R has the *lossless* (*nonadditive*) *join* property *with respect to a set of functional dependencies F* on R if and only if *either*:

- The FD $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+, *or*
- The FD $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+.

In the previous examples, do their decompositions possess the *lossless (nonadditive) join property*?

5.5. Properties of Relational Decompositions

Test the *lossless* (*nonadditive*) *join* property of a *binary decomposition*

A decomposition D = $\{R_1, R_2\}$ of R … *either*:

- The FD $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+, *or*
- The FD $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+.

BEFORE

- Relation: TEACH (STUDENT, COURSE, INSTRUCTOR)
  - Primary key: STUDENT, COURSE
  - Functional dependencies
    - FD1: {STUDENT, COURSE} → INSTRUCTOR
    - FD2: INSTRUCTOR → COURSE

DECOMPOSITION 1

- Relation: STUDENT_COURSE (STUDENT, COURSE)
  - Primary key: STUDENT, COURSE
- Relation: COURSE_INSTRUCTOR (COURSE, INSTRUCTOR)
  - Primary key: INSTRUCTOR
  - Functional dependency: INSTRUCTOR → COURSE

COURSE → STUDENT

COURSE → INSTRUCTOR

DECOMPOSITION 2

- Relation: STUDENT_INSTRUCTOR (STUDENT, INSTRUCTOR)
  - Primary key: STUDENT, INSTRUCTOR
- Relation: COURSE_INSTRUCTOR (COURSE, INSTRUCTOR)
  - Primary key: INSTRUCTOR
  - Functional dependency: INSTRUCTOR → COURSE

INSTRUCTOR → STUDENT

INSTRUCTOR → COURSE

5.6. Algorithms for Relational Database Schema Design

- Algorithm for *dependency-preserving* decomposition into **3NF** schemas
- Algorithm for *dependency-preserving and nonadditive* (*lossless*) *join* decomposition into **3NF** schemas
- Algorithm for *lossless* (*nonadditive*) *join* decomposition into **BCNF** schemas

5.6. Algorithms for Relational Database Schema Design

Algorithm for **dependency-preserving** decomposition into **3NF** schemas

Algorithm Relational Synthesis into 3NF with Dependency Preservation

Input: A universal relation R and a set of functional dependencies F on the attributes of R.

1. Find a minimal cover G for F
2. For each left-hand-side X of a functional dependency that appears in G, create a relation schema in D with attributes $\{X \cup \{A_1\} \cup \{A_2\} \ldots \cup \{A_k\}\}$, where $X \rightarrow A_1$, $X \rightarrow A_2, \ldots, X \rightarrow A_k$ are the only dependencies in G with X as the left-hand-side (X is the key of this relation);
3. Place any remaining attributes (that have not been placed in any relation) in a single relation schema to ensure the attribute preservation property.

5.6. Algorithms for Relational Database Schema Design

Algorithm for **dependency-preserving** and **nonadditive (lossless) join** decomposition into **3NF** schemas

Algorithm Relational Synthesis into 3NF with Dependency Preservation and Nonadditive (Lossless) Join Property

Input: A universal relation R and a set of functional dependencies F on the attributes of R.

1. Find a minimal cover G for F
2. For each left-hand-side X of a functional dependency that appears in G create a relation schema in D with attributes $\{X \cup \{A_1\} \cup \{A_2\} \ldots \cup \{A_k\}\}$, where $X \rightarrow A_1, X \rightarrow A_2, \ldots, X \rightarrow A_k$ are the only dependencies in G with X as left-hand-side (X is the key of this relation).
3. If none of the relation schemas in D contains a key of R, then create one more relation schema in D that contains attributes that form a key of R.

Algorithms for Relational Database Schema Design

Algorithm for **dependency-preserving** and **nonadditive (lossless) join** decomposition into **3NF** schemas

Given a relation schema R = (A, B, C, D, G, H, I, J) and a set E of functional dependencies as follows:

E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}

Perform *dependency-preserving and nonadditive (lossless) join decomposition on R* into 3NF schemas.

Find the minimal cover: F = {A->B, ACD->I, IJ->G, IJ->H}

Find all keys: Key = {A, C, D, J}

Start normalization: R (A, B, C, D, G, H, I, J)

F = {A->B, ACD->I, IJ->G, IJ->H}

Key = {A, C, D, J}

Algorithms for Relational Database Schema Design

Algorithm for **dependency-preserving** and **nonadditive (lossless) join** decomposition into **3NF** schemas

Given a relation schema R = (A, B, C, D, G, H, I, J) and a set E of functional dependencies as follows:

E = {A → B, ABCD → I, IJ → G, IJ → H, ACDJ → GI}

Perform *dependency-preserving and nonadditive (lossless) join decomposition on R* into 3NF schemas.

Identify the current normal form: 1NF because of: A->B, ACD->I

Normalization according to the algorithm:

R1 (A, B)

R2 (A, C, D, I)

R3 (I, J, G, H)

Added for key: R4 (A, C, D, J)

5.6. Algorithms for Relational Database Schema Design

Algorithm for **lossless (nonadditive) join** decomposition into **BCNF** schemas

Algorithm Relational Decomposition into BCNF with Nonadditive Join Property

Input: A universal relation R and a set of functional dependencies F on the attributes of R.

1. Set $D := \{R\}$;
2. While there is a relation schema Q in D that is not in BCNF do
   {
   choose a relation schema Q in D that is not in BCNF;
   find a functional dependency $X \rightarrow Y$ in Q that violates BCNF;
   replace Q in D by two relation schemas $(Q - Y)$ and $(X \cup Y)$;
   };

5.6. Algorithms for Relational Database Schema Design

Algorithm for **lossless (nonadditive) join** decomposition into **BCNF** schemas

Given a relation schema R = (A, B, C, D, E) and a set F of functional dependencies as follows:

$$F = \{A \rightarrow BC,\ CD \rightarrow E,\ B \rightarrow D,\ E \rightarrow A\}$$

Perform *nonadditive (lossless) join decomposition on R* into BCNF schemas.

Find keys of R $\Rightarrow$ K = {A}, {B, C}, {C, D}, {E}

Create a schema from functional dependencies with each distinct X on the left side that violate BCNF:

R1 = (B, D), Key1 = {B},

F1 = {B → D}

R2 = (A, B, C, E), Key2 = {A}, {B, C}, {E},

F2 = {A → BCE, BC → AE, E → ABC}

Lost FD = {CD → E}

5.7. Multivalued Dependencies and Fourth Normal Form

- Multivalued dependencies
  - A multivalued dependency $X \twoheadrightarrow Y$ specified on relation schema R, where X and Y are both subsets of R, specifies the following constraint on any relation state r of R: if two tuple t_1 and t_2 exist in r such that $t_1[X] = t_2[X]$, then two tuples t_3 and t_4 should also exist in r with the following properties, where Z is used to denote (R-(XUY)):
    - $t_3[X] = t_4[X] = t_1[X] = t_2[X]$
    - $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$
    - $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$

5.7. Multivalued Dependencies and Fourth Normal Form

- Multivalued dependencies

EMP

ENAME	PNAME	DNAME
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

  - ENAME $\twoheadrightarrow$ PNAME
  - ENAME $\twoheadrightarrow$ DNAME
  - No association between PNAME and DNAME

5.7. Multivalued Dependencies and Fourth Normal Form

- Multivalued dependencies (MVDs)
  - A multivalued dependency $X \twoheadrightarrow Y$ is a **trivial** MVD if:
    - (a) Y is a subset of X
    - (b) X U Y = R

    → A trivial MVD does not specify any significant or meaningful constraint on R.
  - A **nontrivial** MVD satisfies neither (a) nor (b).

5.7. Multivalued Dependencies and Fourth Normal Form

- Fourth normal form
  - A relation schema R is in 4NF with respect to a set of dependencies (that includes functional dependencies and multivalued dependencies) if, for every nontrivial multivalued dependency $X \twoheadrightarrow Y$ in F^+, X is a superkey for R.

EMP

ENAME	PNAME	DNAME
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

MVDs: ENAME $\twoheadrightarrow$ PNAME
ENAME $\twoheadrightarrow$ DNAME

BCNF

EMP_PROJECTS

ENAME	PNAME
Smith	X
Smith	Y

EMP_DEPENDENTS

ENAME	DNAME
Smith	John
Smith	Anna

4NF

Summary

- Database design process from conceptual modeling:
  - Conceptual database design
    - The entity-relationship model
  - Choice of a DBMS → a representational data model
    - The relational data model
  - Data model mapping for a logical database schema
- Database design process from synthesis and normalization
- Functional dependencies and multivalued dependencies (constraints)

Summary

- Normalization: 1NF, 2NF, 3NF, BCNF
  - Further reading: 4NF, 5NF, 6NF
- Properties of relational decompositions
  - Attribute preservation property (*always required*)
  - Dependency preservation property
  - Lossless (nonadditive) join property
- Algorithm for dependency-preserving decomposition into 3NF schemas
- Algorithm for dependency-preserving and nonadditive (lossless) join decomposition into 3NF schemas
- Algorithm for lossless (nonadditive) join decomposition into BCNF schemas

Chapter 5: Relational Database Design

Review

- 1. Discuss attribute semantics as an informal measure of goodness for a relation schema. Give an example.
- 2. Discuss insertion, deletion, and modification anomalies. Why are they considered bad? Illustrate with examples.
- 3. Why should NULLs in a relation be avoided as much as possible? Illustrate with examples.
- 4. Discuss the problem of spurious tuples and how we may prevent it. Give an example.
- 5. State the informal guidelines for relation schema design that we discussed. Illustrate how violation of these guidelines may be harmful.

Review

- 6. What is a functional dependency? What are the possible sources of the information that defines the functional dependencies that hold among the attributes of a relation schema?
- 7. Define first, second, and third normal forms when only primary keys are considered. How do the general definitions of 2NF and 3NF, which consider all keys of a relation, differ from those that consider only primary keys?
- 8. What undesirable dependencies are avoided when a relation is in 2NF?
- 9. What undesirable dependencies are avoided when a relation is in 3NF?
- 10. In what way do the generalized definitions of 2NF and 3NF extend the definitions beyond primary keys?
- 11. Define Boyce-Codd normal form. How does it differ from 3NF? Why is it considered a stronger form of 3NF?

Review

- 12. What is meant by the completeness and soundness of Armstrong's inference rules?
- 13. What is meant by the closure of a set of functional dependencies? Illustrate with an example.
- 14. When are two sets of functional dependencies equivalent? How can we determine their equivalence?
- 15. What is a minimal set of functional dependencies? Does every set of dependencies have a minimal equivalent set? Is it always unique?
- 16. What is meant by the attribute preservation condition on a decomposition?
- 17. What is the dependency preservation property for a decomposition? Why is it important?
- 18. What is the lossless (or nonadditive) join property of a decomposition? Why is it important?
- 19. Between the properties of dependency preservation and losslessness, which one must definitely be satisfied? Why?

Review

20. Suppose that we have the following requirements for a university database that is used to keep track of students' transcripts:

a. The university keeps track of each student's name (Sname), student number (Snum), Social Security number (Ssn), current address (Sc_addr) and phone (Sc_phone), permanent address (Sp_addr) and phone (Sp_phone), birth date (Bdate), sex (Sex), class (Class) ('freshman', 'sophomore', … , 'graduate'), major department (Major_code), minor department (Minor_code) (if any), and degree program (Prog) ('b.a.', 'b.s.', … , 'ph.d.'). Both Ssn and student number have unique values for each student.

b. Each department is described by a name (Dname), department code (Dcode), office number (Doffice), office phone (Dphone), and college (Dcollege). Both name and code have unique values for each department.

c. Each course has a course name (Cname), description (Cdesc), course number (Cnum), number of semester hours (Credit), level (Level), and offering department (Cdept). The course number is unique for each course.

d. Each section has an instructor (Iname), semester (Semester), year (Year), course (Sec_course), and section number (Sec_num). The section number distinguishes different sections of the same course that are taught during the same semester/year; its values are 1, 2, 3, … , up to the total number of sections taught during each semester.

e. A grade record refers to a student (Ssn), a particular section, and a grade (Grade).

Design a relational database schema for this database application. First show all the functional dependencies that should hold among the attributes. Then design relation schemas for the database that are each in 3NF or BCNF. Specify the key attributes of each relation. Note any unspecified requirements, and make appropriate assumptions to render the specification complete.

Review

- 21. Consider the relation DISK_DRIVE (Serial_number, Manufacturer, Model, Batch, Capacity, Retailer). Each tuple in the relation DISK_DRIVE contains information about a disk drive with a unique Serial_number, made by a manufacturer, with a particular model number, released in a certain batch, which has a certain storage capacity and is sold by a certain retailer. For example, the tuple Disk_drive ('1978619', 'WesternDigital', 'A2235X', '765234', 500, 'CompUSA') specifies that WesternDigital made a disk drive with serial number 1978619 and model number A2235X, released in batch 765234; it is 500GB and sold by CompUSA.

Write each of the following dependencies as an FD:

a. The manufacturer and serial number uniquely identifies the drive.

b. A model number is registered by a manufacturer and therefore can't be used by another manufacturer.

c. All disk drives in a particular batch are the same model.

d. All disk drives of a certain model of a particular manufacturer have exactly the same capacity.

Review

22. Consider the following relation:

A	B	C	TUPLE#
10	b1	c1	1
10	b2	c2	2
11	b4	c1	3
12	b3	c4	4
13	b1	c1	5
14	b3	c4	6

a. Given the previous extension (state), which of the following dependencies *may hold* in the above relation? If the dependency cannot hold, explain why *by specifying the tuples that cause the violation.*

i. $A \rightarrow B$, ii. $B \rightarrow C$, iii. $C \rightarrow B$, iv. $B \rightarrow A$, v. $C \rightarrow A$

b. Does the above relation have a potential candidate key? If it does, what is it? If it does not, why not?

Review

- 23. Consider the universal relation R = {A, B, C, D, E, F, G, H, I, J} and the set of functional dependencies F = {{A, B}→{C}, {A}→{D, E}, {B}→{F}, {F}→{G, H}, {D}→{I, J}}. What is the key for R? Decompose R into 2NF, then 3NF and BCNF relations.
- 24. For the following different set of functional dependencies G = {{A, B}→{C}, {B, D}→{E, F}, {A, D}→{G, H}, {A}→{I}, {H}→{J}}. What is the key for R? Decompose R into 2NF, then 3NF and BCNF relations.
- 25. Consider a relation R(A, B, C, D, E) with the following dependencies:

$$AB \rightarrow C,\ CD \rightarrow E,\ DE \rightarrow B$$

Is AB a candidate key of this relation? If not, is ABD? Explain your answer. How would you normalize this relation?

Review

- 26. Consider the relation R, which has attributes that hold schedules of courses and sections at a university; R = {Course_no, Sec_no, Offering_dept, Credit_hours, Course_level, Instructor_ssn, Semester, Year, Days_hours, Room_no, No_of_students}. Suppose that the following functional dependencies hold on R:

{Course_no} → {Offering_dept, Credit_hours, Course_level}

{Course_no, Sec_no, Semester, Year} → {Days_hours, Room_no, No_of_students, Instructor_ssn}

{Room_no, Days_hours, Semester, Year} → {Instructor_ssn, Course_no, Sec_no}

Try to determine which sets of attributes form keys of R. How would you normalize this relation?

Review

- 27. Consider the following relation:

CAR_SALE(Car#, Date_sold, Salesperson#, Commission%, Discount_amt)

Assume that a car may be sold by multiple salespeople, and hence {Car#, Salesperson#} is the primary key. Additional dependencies are:

Date_sold → Discount_amt

Salesperson# → Commission%

Based on the given primary key, is this relation in 1NF, 2NF, 3NF, or BCNF? Why or why not? How would you successively normalize it completely?

Review

- 28. Consider the following relation for published books:

BOOK (Book_title, Author_name, Book_type, List_price, Author_affil, Publisher)

Author_affil refers to the affiliation of author. Suppose the following dependencies exist:

Book_title → Publisher, Book_type

Book_type → List_price

Author_name → Author_affil

a. What normal form is the relation in? Explain your answer.

b. Apply normalization until you cannot decompose the relations further. State the reasons behind each decomposition.

Review

- 29. Consider the following relation:

  R (Doctor#, Patient#, Date, Diagnosis, Treat_code, Charge)

In the above relation, a tuple describes a visit of a patient to a doctor along with a treatment code and daily charge. Assume that diagnosis is determined (uniquely) for each patient by a doctor. Assume that each treatment code has a fixed charge (regardless of patient). Is this relation in 2NF? Justify your answer and decompose if necessary. Then argue whether further normalization to 3NF is necessary, and if so, perform it.

Review

- 30. Consider the following relation:

CAR_SALE (Car_id, Option_type, Option_listprice, Sale_date, Option_discountedprice)

This relation refers to options installed in cars (e.g., cruise control) that were sold at a dealership, and the list and discounted prices of the options.

If CarID → Sale_date

and Option_type → Option_listprice

and CarID, Option_type → Option_discountedprice,

argue using the generalized definition of the 3NF that this relation is not in 3NF. Then argue from your knowledge of 2NF, why it is not even in 2NF.

Review

- 31. Consider the relation REFRIG(Model#, Year, Price, Manuf_plant, Color), which is abbreviated as REFRIG(M, Y, P, MP, C), and the following set F of functional dependencies: F = {M → MP, {M, Y} → P, MP → C}

a. Evaluate each of the following as a candidate key for REFRIG, giving reasons why it can or cannot be a key: {M}, {M, Y}, {M, C}.

b. Based on the above key determination, state whether the relation REFRIG is in 3NF and in BCNF, and provide proper reasons.

c. Consider the decomposition of REFRIG into D = {R1(M, Y, P), R2(M, MP, C)}. Is this decomposition lossless? Show why.

Review

- 32. Consider the following decompositions for the relation schema R = {A, B, C, D, E, F, G, H, I, J} with the following functional dependencies:

F = {{A, B}→{C}, {A}→{D, E}, {B}→{F}, {F}→{G, H}, {D}→{I, J}}.

Determine whether each decomposition has (1) the dependency preservation property, and (2) the lossless join property, with respect to F.

Also determine which normal form each relation in the decomposition is in.

a. D1 = {R1, R2, R3, R4, R5}; R1 = {A, B, C}, R2 = {A, D, E}, R3 = {B, F}, R4 = {F, G, H}, R5 = {D, I, J}

b. D2 = {R1, R2, R3}; R1 = {A, B, C, D, E}, R2 = {B, F, G, H}, R3 = {D, I, J}

c. D3 = {R1, R2, R3, R4, R5}; R1 = {A, B, C, D}, R2 = {D, E}, R3 = {B, F}, R4 = {F, G, H}, R5 = {D, I, J}

Next

Chapter 6: Physical Storage - Data Management

- 6.1. Physical Data Storage
- 6.2. Indexing
- 6.3. Complex Data Management Approaches (Semi-structured and Unstructured Data)
- 6.4. Massive Data Management Approaches
- 6.5. Quality Issues: Reliability, Scalability, Effectiveness, Efficiency